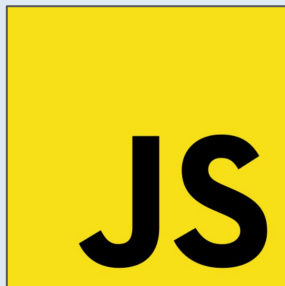




Institut TIC
de Barcelona



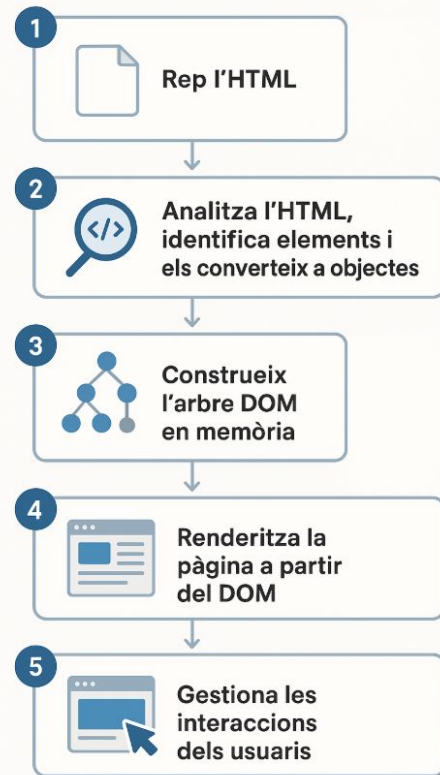
RA6.FE - Document Object Model (DOM)

- DOM és una API del navegador que s'encarrega de guardar en **memòria** la representació estructurada del document HTML en forma d'**arbre**.
- Permet a javascript accedir i modificar el contingut, l'estructura i l'estil d'una pàgina de manera dinàmica.



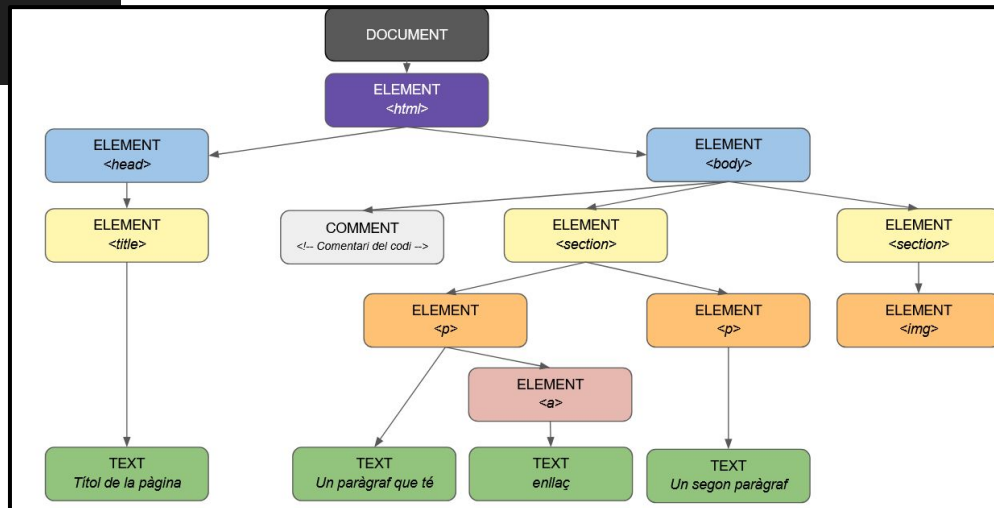
http/s

Navegador



```
<!DOCTYPE html>
<html>
  <head>
    <title>Títol de la pàgina</title>
  </head>
  <body>
    <!-- Comentari del codi -->
    <section>
      <p>Un paràgraf que té <a href="#">enllaç</a></p>
      <p>Un segon paràgraf</p>
    </section>
    <section>
      
    </section>
  </body>
</html>
```

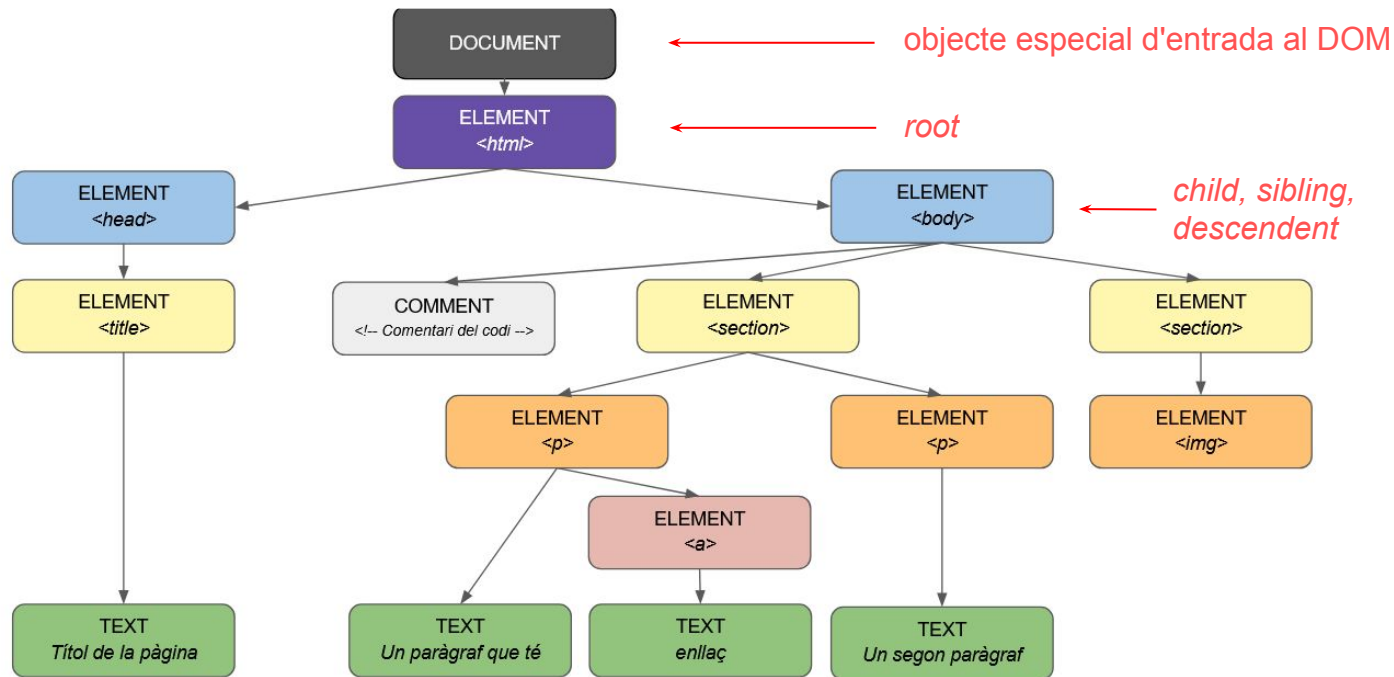
Parsejar un document html a una estructura d'arbre del DOM.



Tots els requadres son **nodes**.

Tenim diferents **tipus de nodes**:

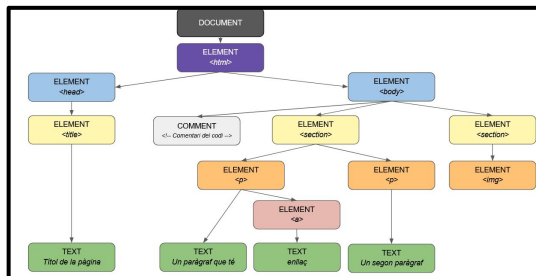
- **Elements**
Les etiquetes html
- **Text**
Els text de les etiquetes
- **Comment**
Son els comentaris
- **Document**
Objecte del document



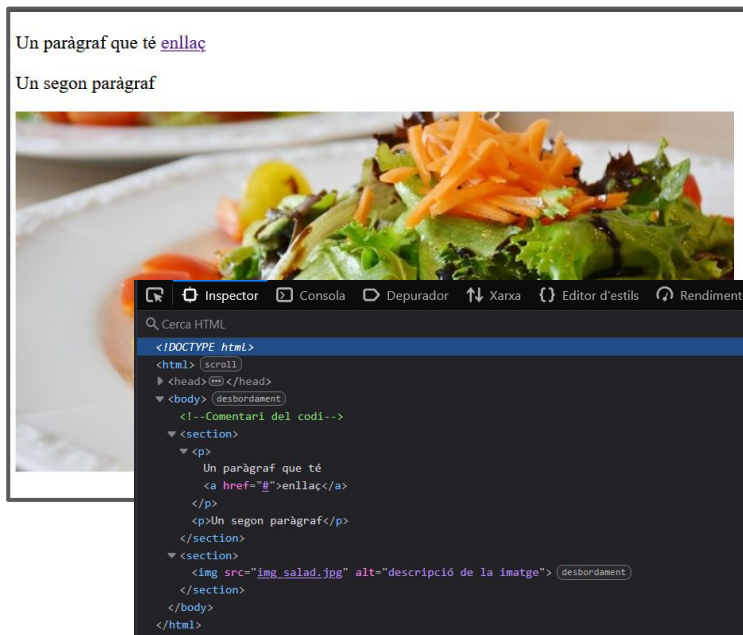
Pàgina html



DOM



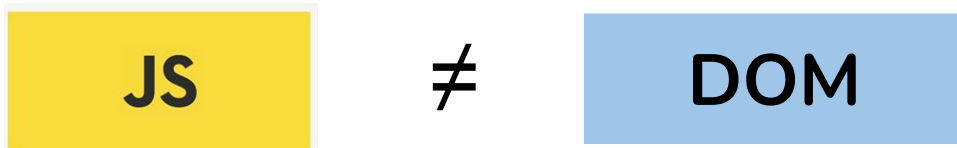
Pantalla navegador



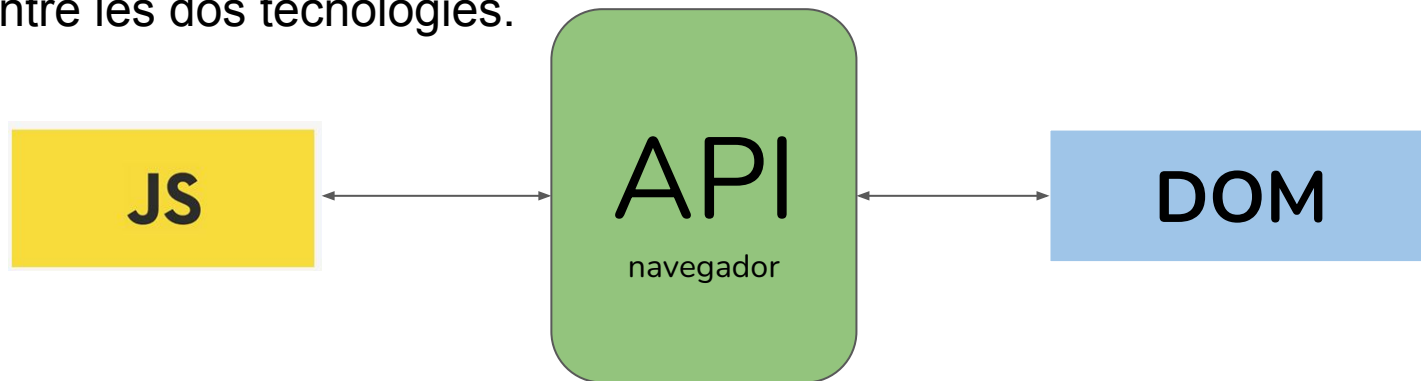
- Es crea el DOM en memòria

- Renderitza el DOM per pantalla
- Si es modifica el DOM, es renderitzen els canvis per pantalla

- Javascript i DOM son tecnologies diferents que no tenen cap relació.



- Javascript utilitza la API del navegador per poder-se comunicar i interaccionar entre les dos tecnologies.



- Per tal de que javascript pugui interactuar amb el DOM s'utilitza una sèries **d'atributs i mètodes** que ens proporciona l'objecte ***document***.
- A partir d'aquest objecte en javascript podem:
 - Seleccionar elements
 - Modificar contingut
 - Manipular atributs
 - Modificar estils CCS inline
 - Treballar amb classes de css
 - Capturar events
 - Crear i eliminar elements
 - Validacions de formularis
 - Animacions

- **querySelector** (seleccionem un element per selector)

```
<header>
  <h1 class="main-tittle"> Hellor world</h1>
</header>
<script>
  //Seleccionar un únic element a través del selectors o combinacions
  const element3 = document.querySelector('header .main-tittle');
  console.log(element3, typeof element3);
  //Retorna una etiqueta
</script>
```

▶ `<h1 class="main-tittle">`  object

- **querySelectorAll** (seleccionem tots els elements per selector)

```
<header>
  <h1 class="main-tittle"> Hellor world</h1>
  <h3 class="main-tittle"> Hellor world</h3>
</header>
<script>
  // Seleccionar elements a través del selectors o combinacions
  const element4 = document.querySelectorAll('header .main-tittle');
  console.log(element4, typeof element4);
  //Retorna NodeList
</script>
```

▼ `NodeList [ h1.main-tittle, h3.main-tittle ]`
▶ 0: `<h1 class="main-tittle">`
▶ 1: `<h3 class="main-tittle">`
length: 2

- id

```
<h1 id="tittle-id"> Hellor world</h1>
<script>
  //Seleccionar un element per l'identificar de l'etiqueta
  const element = document.getElementById('tittle-id');
  console.log(element, typeof element);
  //Retorna l'etiqueta
</script>
```

▶ `<h1 id="tittle-id">`  object

- classe

```
<h1 class="main-tittle"> Hellor world</h1>
<script>
  //Seleccionar un element per l'identificar de l'etiqueta
  const element2 = document.getElementsByClassName('main-tittle');
  console.log(element2, typeof element2);
  //Retorna HTMLCollection
</script>
```

▶ `HTMLCollection { 0: h1.main-tittle, length: 1 }` object

- **textContent**

```
<header>
|   <h1 class="main-tittle"> Hellor world</h1>
</header>
<script>
|   const titleObj = document.querySelector('header .main-tittle');
|   // Amb textContent podem canviar el contingut de l'element
|   titleObj.textContent="Nou text que afegim";
</script>
```

- **innerHTML** (menys segur)

```
<header>
|   <h1 class="main-tittle"> Hellor world</h1>
</header>
<script>
|   const titleObj = document.querySelector('header .main-tittle');
|   // Amb innerHTML podem canviar el contingut de l'element i afegir etiquetes.
|   titleObj.innerHTML="Nou text <em> amb etiquetes HTML</em>";
</script>
```

- **innerText**

```
<header>
|   <h1 class="main-tittle"> Hellor world</h1>
</header>
<script>
|   const titleObj = document.querySelector('header .main-tittle');
|   // Amb innerText podem afegir contingut text:
|   titleObj.innerText ='Nou text ';
</script>
```

- Modificar imatges

```

<script>
  const img = document.querySelector('img');
  img.src = 'img2.jpg';
  img.alt = 'Nova descripció de la imatge';
  img.id = 'imatge-principal';
</script>
```

- Modificar enllaços

Actualment s'utilitza més el **getAttribute** i **setAttribute** per modificar els atributs

```
<a href="https://www.google.es">Enllaç a google</a>
<script>
  const link = document.querySelector('a');
  // Llegir atributs
  const href = link.getAttribute('href');
  // Modificar atributs
  link.setAttribute('target', '_blank');
  // Eliminar atributs
  link.removeAttribute('title');
</script>
```

- Per modificar els estils hem d'utilitzar l'atribut **style**. Modifiquem els estils de forma inline sobre el mateix element.

```
<p>Hello world</p>
<script>
  const element = document.querySelector('p');

  element.style.color = 'red';
  element.style.backgroundColor = '#f0f0f0';
  element.style.fontSize = '30px';

  element.style.setProperty('background-color', 'blue');
  element.style.setProperty('padding-top', '15px');
  element.style.setProperty('padding-bottom', '15px');
</script>
```

Actualment s'utilitza més el **getAttribute** i **setAttribute** per modificar els atributs

- Per modificar els estils hem d'utilitzar l'atribut **style**. Modifiquem els estils de forma inline sobre el mateix element.

```
<p class="par1">Text1</p>
<p class="par2">Text2</p>
<p class="par3">Text3</p>

<script>
  const pText2Obj = document.querySelector('.par2');
  const pText3Obj = document.querySelector('.par3');

  pText2Obj.style.setProperty('display', 'none');
  pText3Obj.style.setProperty('visibility', 'hidden');

  //pText2Obj.style.display = 'none';
  //pText3Obj.style.visibility = 'hidden';
</script>
```

Recorda!

Display: none;

Existeix l'element però no es renderitza per pantalla. (No ocupa espai)

Visibility: hidden;

Existeix l'element, es renderitza per pantalla però és invisible. (Ocupa l'espai que té reservat)

- Podem **afegir o treure classes** d'estils als elements.
- Les classes ja han d'existir.
- Utilitzant propietat **className**
(No aconsellable)
- Utilitzant **classList**, ja que permet gestionar-ho a través d'una llista.

```
<p> Hello world</p>
<script>
  const p = document.querySelector('p');
  // Assignar directament
  p.className = 'color1 text1';
  // Afegir sense eliminar existents
  p.className += ' classe-extra';
</script>
```

```
<p> Hello world</p>
<script>
  const p = document.querySelector('p');

  // Aplicar dos classes a l'element
  p.classList.add('color1', 'text1');

  // Treure una classe de l'element
  p.classList.remove('color1');

  // Commutar (Afegir si no s'està aplicant / treure si s'aplica)
  p.classList.toggle('visible');

  // Comprovar que un element utilitzi una classe
  if (p.classList.contains('text1')) {
    console.log('Està actiu');
  }

  // Reemplaçar
  p.classList.replace('text1', 'text2');
</script>
```

- Podem **afegir o treure events** i les podem relacionar amb funcions.

```
<button class="btn-play"> Jugar </button>

<script>
  const btnPlayObj = document.querySelector("button");

  // Afegim un event click al botó relacionat amb la funció jugar
  btnPlayObj.addEventListener('click', function(){
    //Cridem una funció de javascript
    jugar();
  });

  // Funció de javascript
  function jugar(){
    console.log('Botó clicat!');
  }
</script>
```

- Podem **afegir elements html** amb javascript amb *createElement*.

```
<!-- div on afegim items -->
<div class="container"></div>
<button class="add-Item">Afegir ítem</button>

<script>
  const btnAddItemObj = document.querySelector(".add-Item");
  const divContainerObj = document.querySelector(".container");

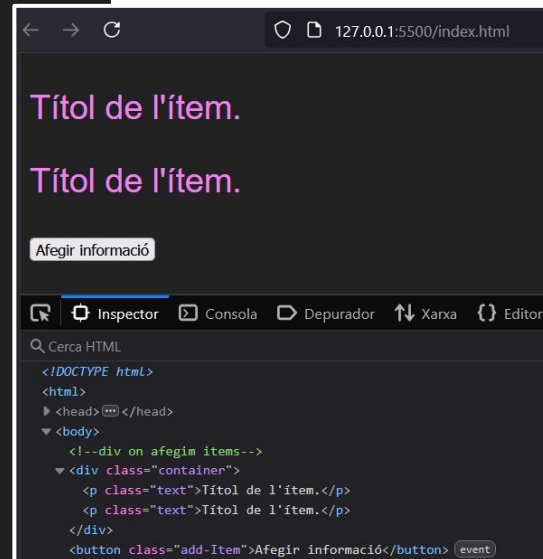
  btnAddItemObj.addEventListener("click", function(){
    afegirItem();
  });

  function afegirItem(){
    //Creem un element html de tipus paràgraf.
    const item = document.createElement("p");

    //Li afegim un text
    item.textContent = "Títol de l'item.";

    //Li afegim estils a través d'una classe.
    //Recorda que la classe ha d'estar creada.
    item.classList.add("text");

    //Afegirem el nou element creat a l'html.
    divContainerObj.appendChild(item);
  }
</script>
```



- Podem **afegir elements html** amb javascript amb ***insertAdjacentX***.

```
<!-- div on afegim items -->
<div class="container"></div>
<button class="add-item">Afegir ítem</button>

<script>
  const btnAddItemObj = document.querySelector(".add-item");
  const divContainerObj = document.querySelector(".container");

  btnAddItemObj.addEventListener("click", function(){
    afegirItem();
  });

  function afegirItem(){
    //Creem un element html de tipus paràgraf.
    const item = document.createElement("p");

    //Li afegim un text
    item.textContent = "Títol de l'ítem.";

    //Li afegim estils a través d'una classe.
    //Recorda que la classe ha d'estar creada.
    item.classList.add("text");

    //Afegirem el nou element creat a l'html.
    divContainerObj.insertAdjacentElement("beforeend", item);
  }
</script>
```

insertAdjacentElement:

Permet afegir elements html. Útil quan has de crear un element amb *createElement*.

insertAdjacentHTML:

Permet afegir text i html conjuntament. Útil quan has d'afegir text amb detalls o estils: *span*, *br*,....

insertAdjacentText:

Permet afegir únicament text.

En tots els casos podem indicar a on volem afegir el nou contingut:

```
<!-- 'beforebegin' -->
<div id="container">
  <!-- 'afterbegin' -->
  Contingut existent
  <!-- 'beforeend' -->
</div>
<!-- 'afterend' -->
```

- Podem **eliminar elements html** amb javascript.

```
<h1 class="tittle">Title</h1>
<div class="container">
  <p class="par">pràgraf1</p>
  <p class="par-del">pràgraf2</p>
  <p class="par">pràgraf3</p>
</div>

<script>

  //Borrar directament l'element
  const hTittleObj = document.querySelector(".tittle");

  hTittleObj.remove();

  //Eliminar un fill del container
  const divContainerObj = document.querySelector(".container");
  const pParagrafObj = document.querySelector(".par-del");

  divContainerObj.removeChild(pParagrafObj);

</script>
```

Compte amb borrar i les propietats display i visibility!

```
const NUM_CLASSES = 4;
const caixaCanvi = document.getElementById("caixa-canvi-estat");
const infoEstat = document.getElementById("info-ciclic");

let comptador = 1;

function canviaCSS() {
    caixaCanvi.classList.remove("estat-1", "estat-2", "estat-3", "estat-4");
    let estat = (comptador % NUM_CLASSES) + 1;
    let nomClass = `estat-${estat}`;
    caixaCanvi.classList.add(nomClass);
    infoEstat.textContent = estat;
    comptador++;
}
```

